

LeetCode 763 - Partition Labels

ALGORITHM DESIGN & INTUITION GUIDE

Problem Statement

We are given a string `s`. We need to divide it into the **maximum possible partitions** such that:

- Every character appears in at most one partition.
- Once a partition is completed, none of its characters should appear again in any subsequent partitions.

The objective is to return a list containing the size of every partition created.

Key Observation

Suppose we have the string `s = "abac"`. Can we partition it like `a | bac` ?

No. Because the character `'a'` appears again later in the string. Before finalizing any partition, we must know: *Where does every character appear for the last time?*

Therefore, the very first step of our solution must be to precompute and store the last occurrence index of every character.

Example Last Occurrences:

```
a → 2
b → 1
c → 3
```

Main Intuition

While traversing the string, we continuously build the current partition. Whenever we encounter a character, we ask: *"When does this character finish?"*

This completion index is given by `last[s[i]]`. Our partition cannot end before this position; otherwise, the character would span across two different partitions, which violates the core constraint.

To implement this, we maintain a running boundary variable:

$$end = \max(end, last[s[i]])$$

This guarantees that every character already included in the current partition is completely contained inside its boundaries.

The Invariant (Most Important Rule)

While traversing the string, the variable `end` always stores **the farthest last occurrence of every character that has appeared in the current partition so far**.

Notice carefully: It does *not* track every character in the entire string globally—only the characters we have actually encountered up to the current index.

Clarifying Doubts & Edge Cases

Doubt 1: What if a character (e.g., 'd') is not inside the current partition?

Example: `ababcbaca | defegde`

While processing the first partition, we only see `'a'`, `'b'`, and `'c'`. We never encounter `'d'`. Consequently, we never look at `last['d']`. A character that hasn't appeared yet places no restriction on the current partition, so we simply ignore it until its turn arrives.

Doubt 2: What if a character appears inside the current partition, but its last occurrence is outside the current boundary?

Example: `a b a c d b e d`

Suppose our initial current boundary is set from index `0` to `5`. Now we encounter `'d'`. The last occurrence of `'d'` is at index `7`. If we were to end the partition at index `5`, `'d'` would be split across two different partitions, which is not allowed.

Thus, we must extend the boundary: `end = max(end, last['d']);`. The new boundary correctly shifts to index `7`. This perfectly illustrates why we always evaluate: `end = max(end, last[s[i]]);`

When Can We Make a Partition?

Suppose `start = 0` and `end = 8`. We keep moving our index `i++` forward. When we reach the point where `i == end`, it means that every character that appeared from `start` to `end` has also finished inside this exact range. Nothing from this partition appears anywhere later in the string.

Therefore, it is now safe to make a cut:

- **Partition Size:** `end - start + 1`
- **Next Partition Start:** `start = end + 1;` (Begin building the next segment).

Why Is This Greedy Strategy Correct?

We always cut at the *earliest possible valid position*. We cannot cut earlier because some character would be left hanging across the boundary. We should not cut later because our goal is to maximize the total number of partitions. Thus, the earliest valid cut is strictly optimal.

Algorithm Steps

Step 1: Precompute Last Occurrences

Store the last occurrence index of every character in an array or hash map.

```
for(int i = 0; i < n; i++) {  
    last[s[i] - 'a'] = i;  
}
```

Step 2: Initialize Pointers

Initialize the partition markers.

```
int start = 0;  
int end = 0;
```

Step 3 & 4: Traverse and Cut

Traverse the string. For every character, update the boundary. When the index matches the boundary, push the size and reset the start pointer.

```
for(int i = 0; i < n; i++) {  
    end = max(end, last[s[i] - 'a']);  
  
    if(i == end) {  
        answer.push_back(end - start + 1);  
        start = i + 1;  
    }  
}
```

Dry Run Example

Given string: $s = \text{"abacd"}$

Last Occurrences: a → 2, b → 1, c → 3, d → 4

Index (i)	Character	Updated End Boundary	Action Taken
0	a	$\max(0, 2) = 2$	Continue tracking current partition.
1	b	$\max(2, 1) = 2$	Continue tracking current partition.
2	a	$\max(2, 2) = 2$	Cut Partition! Size = $2 - 0 + 1 = 3$. Reset start = 3.
3	c	$\max(0, 3) = 3$	Cut Partition! Size = $3 - 3 + 1 = 1$. Reset start = 4.
4	d	$\max(0, 4) = 4$	Cut Partition! Size = $4 - 4 + 1 = 1$. End of string.

Final Answer Output: [3, 1, 1]

Pattern to Recognize

The Greedy + Interval Pattern

Whenever a problem states or implies:

- Divide a sequence into segments/subarrays.
- Maximize or minimize the total number of partitions.
- Every element must belong to exactly one partition.
- Elements/characters/values should not cross over partition boundaries.

Think: Can I precompute the last occurrence (or valid interval range) of every element and greedily extend the current segment until all active intervals are successfully closed?

One-Line Revision for Interviews

Core Summary

Store the last occurrence of every character. While scanning the string, maintain the farthest last occurrence (`end`) of all characters seen in the current partition. When the current index reaches `end` , every character in that partition has finished appearing, allowing a safe, optimal cut that maximizes total partitions.